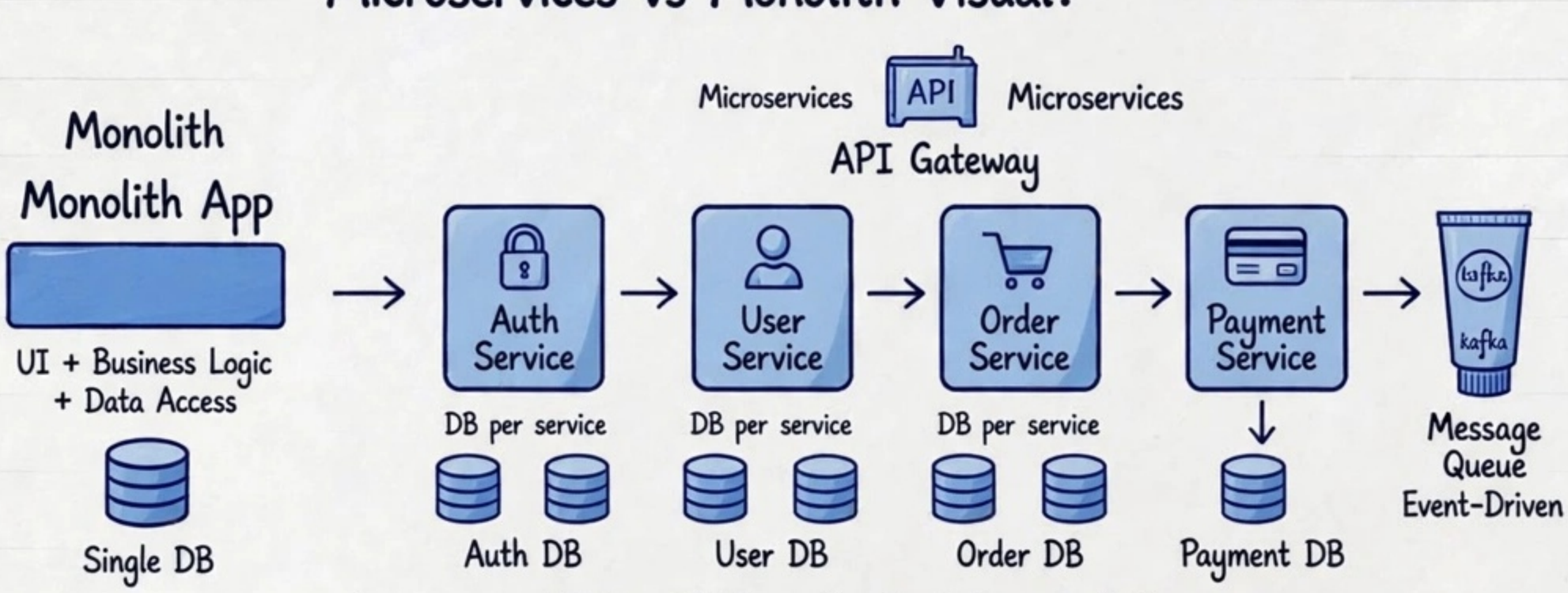
TOP 50 MICROSERVICES INTERVIEW Q&A

(Backend Series) Real Production Scenarios + Visual Learning

Saga • CQRS • Circuit Breaker • Service Discovery • 50 Q&A

Master Microservices Architecture

Microservices vs Monolith Visual:



Communication: REST • gRPC (arrows) + Event-driven via Kafka

- Monolith: single codebase vs Microservices: independent deploy and scale • API Gateway + Service Discovery: Eureka/Consul for service registry •
- DB per service pattern: each service owns its own database
- Event-driven via Kafka / RabbitMQ for async communication
- Scaling: services scale independently based on load

Why Learn This?:

- Most asked in FAANG backend interviews 80%
- Microservices = 90% companies now standard
- Combines LLD + HLD + production issues/ problem salving
- Every senior interview asks saga, circuit breaker, CQRS

In this handbook you'll learn:

1. Microservices vs Monolith Q&A
2. Service Discovery – Eureka Consul
3. API Gateway Pattern
4. Circuit Breaker – Resilience4j Hystrix
5. Saga Pattern – Distributed Transactions
6. CQRS + Event Sourcing
7. Event-Driven Architecture – Kafka
8. Database per Service + Shared DB
9. Distributed Tracing + Logging
10. Load Balancing in Microservices
11. Security – JWT Oauth2 in Microservices
12. Deployment – Docker K8s Helm
13. Scaling + Autoscaling Strategies
14. Microservices vs Serverless Comparison
15. Final Cheatsheet + Top 10 Interview Q&A + Diagram

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

Microservices vs Monolith Q&A

Q1 • Q2 • Q3 • Comparison Table • Diagram

Q1: What is Monolith?

Text: Single codebase, single database.
UI + Business Logic + Data Access in one unit.
Simple to start but hard to scale.

Q2: What is Microservices?

Text: Small independent services, each owns its own DB (DB per service).
Communicates via REST / gRPC / Kafka.
Independently deployable & scalabe.

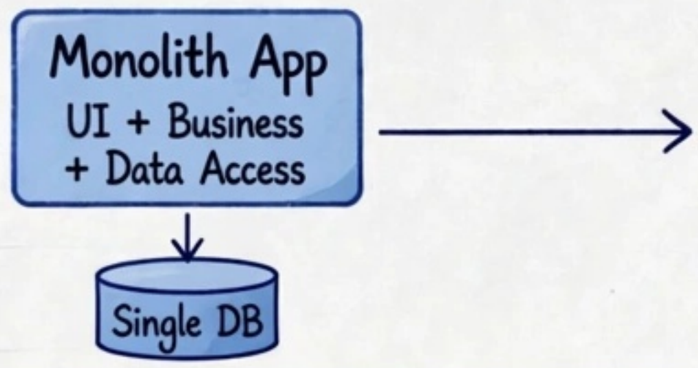
Q3: When to Use? Monolith → Small team, MVP, low complexity | Microservices → Large scale, multiple teams, independent scaling & deployment

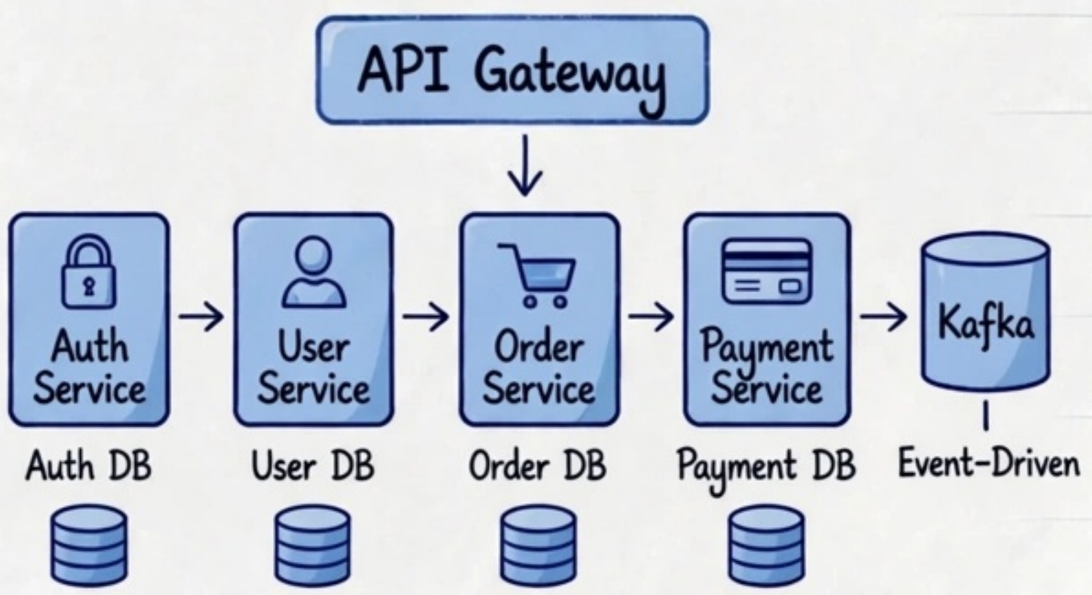
Comparison Table:

Aspect	Monolith	Microservices
Scalability	Vertical vs	Horizontal
Deployment	Single vs	Independent
Tech Stack	Single vs	Polyglot
Database	Single DB vs	DB per service
Failure	Single point vs	Fault isolated
Team	Tight coupling vs	Decoupled

Pros: Monolith simpler to develop & debug; Microservices scalable & flexible.
Cons: Monolith harder to scale; Microservices add complexity.

Diagram:





💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

SERVICE DISCOVERY

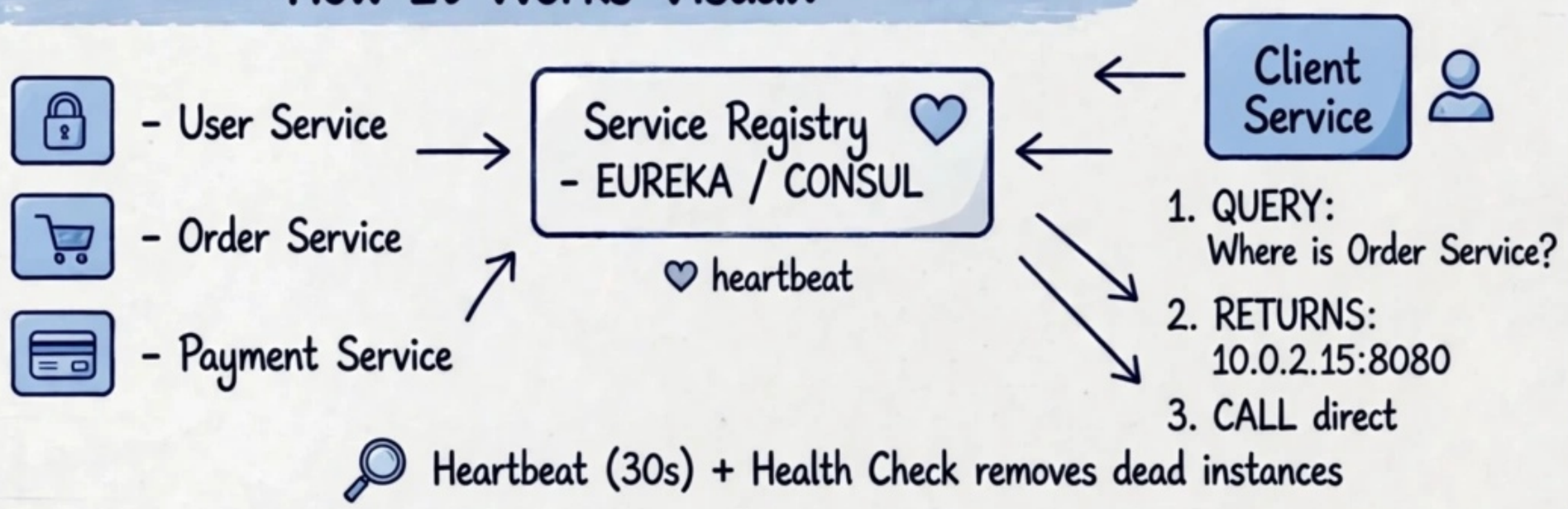
EUREKA • CONSUL • Interview Q&A

How Services Find Each Other - Dynamic IPs Problem

Q1: What is Service Discovery? Why Needed?

- In microservices IPs change constantly (auto-scaling, redeploy)
- Hardcoded IPs impossible
- Service Discovery = automatic detection of service instances

How It Works Visual:



Client-Side Discovery (Eureka)

- Client queries registry
- Client does load balancing
- Example: Spring Cloud Eureka

Eureka vs Consul Quick Comparison

- Eureka: AP (Availability), Client-side, Java / Spring native, Self-preservation mode
- Consul: CP (Consistency), KV store, Health checks, Multi-DC, Server-Side + Client-Side both

Top Interview Q&A

Q: What if Eureka dies? A: Clients cache registry
Q: How Consul health check? A: HTTP / TTL / Script checks
Q: Service Registry vs API Gateway?
A: Registry = who & where, Gateway = entry + routing

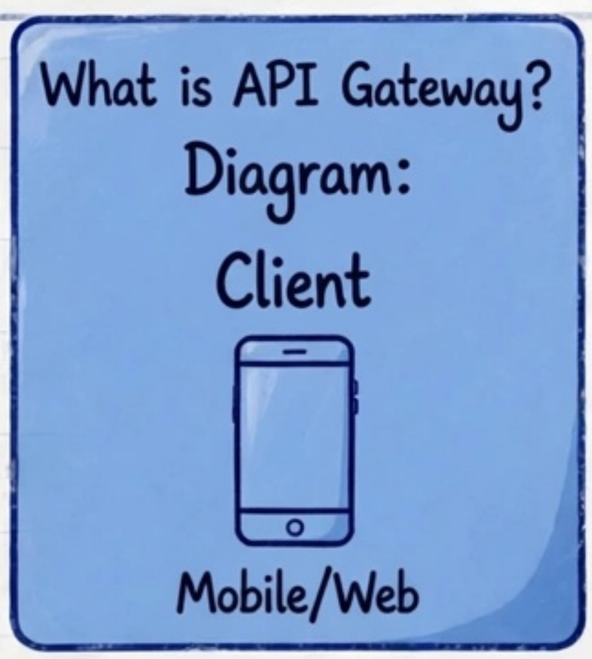
Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

Q3) API GATEWAY PATTERN - Single Entry Point?

(Most Asked in System Design + Backend Interviews)

What is API Gateway?

Diagram:



Communication: Client → Gateway → Services

Popular Implementations:

- Kong (Nginx based, plugin ecosystem)
  Netflix Zuul (JVM based, filter based)
- Routing: /api/users → User Service, /api/orders → Order Service
- Cross-cutting: Authentication, SSL termination, Rate Limiting, Logging
- Aggregation: combine multiple service results into one response
- BFF - Backend for Frontend pattern: separate gateway for Web vs Mobile vs 3rd Party

Interview Q&A:

Q: Gateway vs Load Balancer?

A: LB distributes traffic, Gateway adds app logic (auth, routing)

Q: If Gateway fails?

A: High Availability, multiple instances + LB in front

Q: BFF advantages?

A: Tailored API per client type, less over-fetching

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

TOP 50 MICROSERVICES INTERVIEW Q&A

Circuit Breaker Pattern - Resilience4j / Hystrix (Backend Series)

Q: What if Payment Service goes down? Will it cause cascade failure?

Without Circuit Breaker -> YES. One slow/down service exhausts threads, causes timeout chain, brings down whole system.
With Circuit Breaker -> NO, it isolates failure, returns fallback instantly.

Circuit Breaker States + Flow:

CLOSED:
Normal - requests pass, counting failures, 🟢 green light 🟢

OPEN:
Failure threshold exceeded, trips open, fast-fail to Fallback, no call to Payment ❗

HALF OPEN:
After timeout, allows few test requests, if success -> Closed, if fail -> Open again

↓

Fallback Response: Default / Cached / Queue for retry - prevents cascade

Why Learn This?:

- Most asked in FAANG backend interviews
- Prevents cascade failure in microservices
- Resilience4j is modern (Hystrix deprecated) - sliding window, time based
- Combines retry + timeout + bulkhead + fallback

Key Configs Interview Must Know:

- failureRateThreshold
- slidingWindowSize
- waitDurationInOpenState
- permittedCallsInHalfOpenState

💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

Q5. SAGA PATTERN

Distributed Transactions Q&A

How to handle transactions across microservices?

Saga Pattern Diagram:

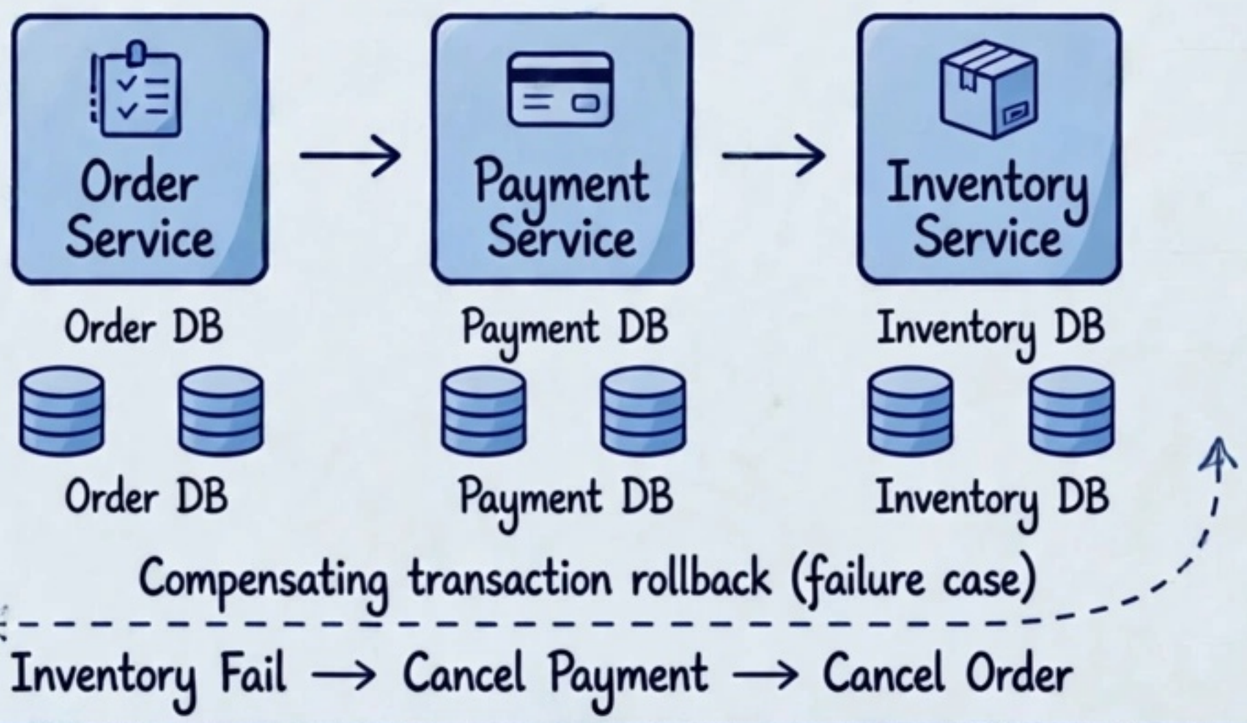
~~Traditional 2PC~~
(Two Phase Commit)

- Not Scalable ~~crosseted~~
- 2PC locks DB, not scalable in microservices

Blocks across services
- Tight coupling

→

Saga solution: sequence of local transactions + compensating transactions



What is Saga?

- Saga = sequence of local transactions, each service does its own local tx + publishes event
- If one step fails, compensating transactions rollback previous steps
- Avoids 2PC, maintains eventual consistency
- Real production: e-commerce Order → Payment → Inventory

Choreography vs Orchestration:

Choreography:
- Services publish events, each reacts (decentralized)
- Uses Kafka / RabbitMQ • Event-driven
- No single point of failure
- Harder to debug

Orchestration:
- Central orchestrator controls the saga (orchestrator service)
- Orchestrator calls services & handles compensations
- Easier to manage & monitor

Compensating Pattern Example:

Order Created → Payment Success → Inventory Reserve Fail → Compensation: Refund Payment + Cancel Order

Note: Compensation must be idompotent & retryable

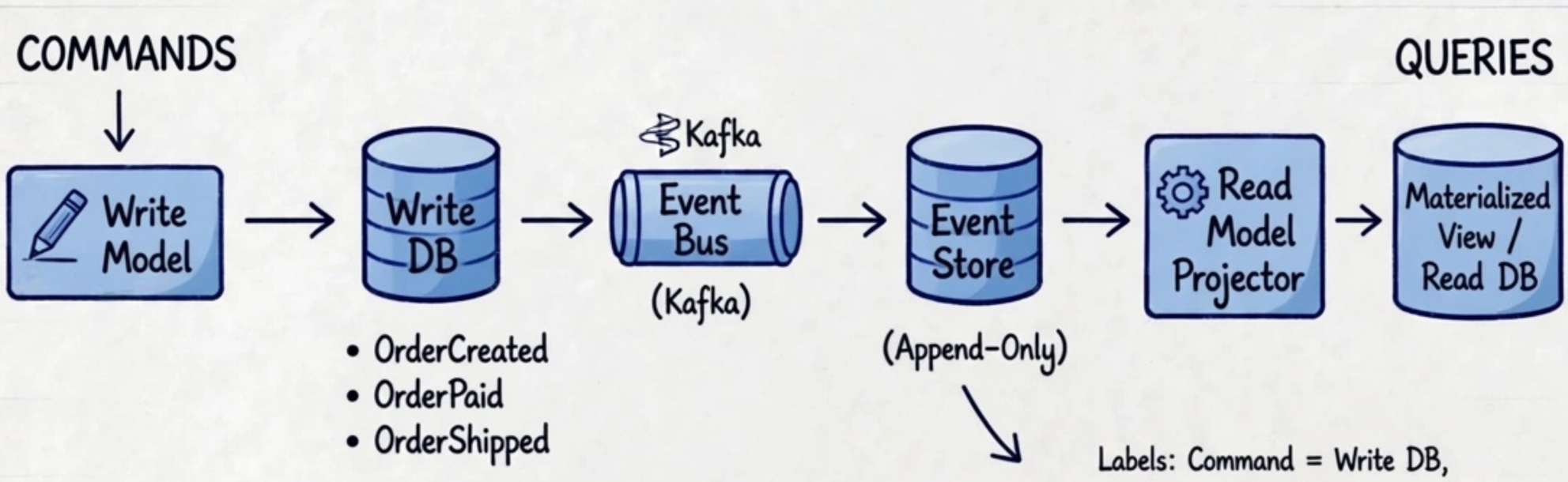
💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

Q07: CQRS + EVENT SOURCING

Command Query Responsibility Segregation Explained

(Backend Series) Separate Read & Write Models + Visual Learning

CQRS + Event Sourcing Visual:



- What is CQRS? Command Query Responsibility Segregation – separate read & write models
- Command: handles writes, updates state, writes to Write DB
- Query: handles reads from optimized Read DB /
- Event Sourcing: store events (facts) not current state, rebuild state by replaying events
- Benefits: Scalability (scale read/write independently), Performance (optimized read models), Audit log, Time-travel, Event-driven

Why CQRS?

- Most asked in system design interviews
- Solves DB bottleneck
- Enables microservices eventual consistency

In this Q&A you'll learn:

1. What is CQRS?
2. Command vs Query
3. What is Event Sourcing?
4. Event Store vs DB
5. CQRS + Event Sourcing together
6. Benefits & Drawbacks
7. When to use / not use
8. Real Production Example: Order Service

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

TOPIC 07 - EVENT-DRIVEN ARCHITECTURE

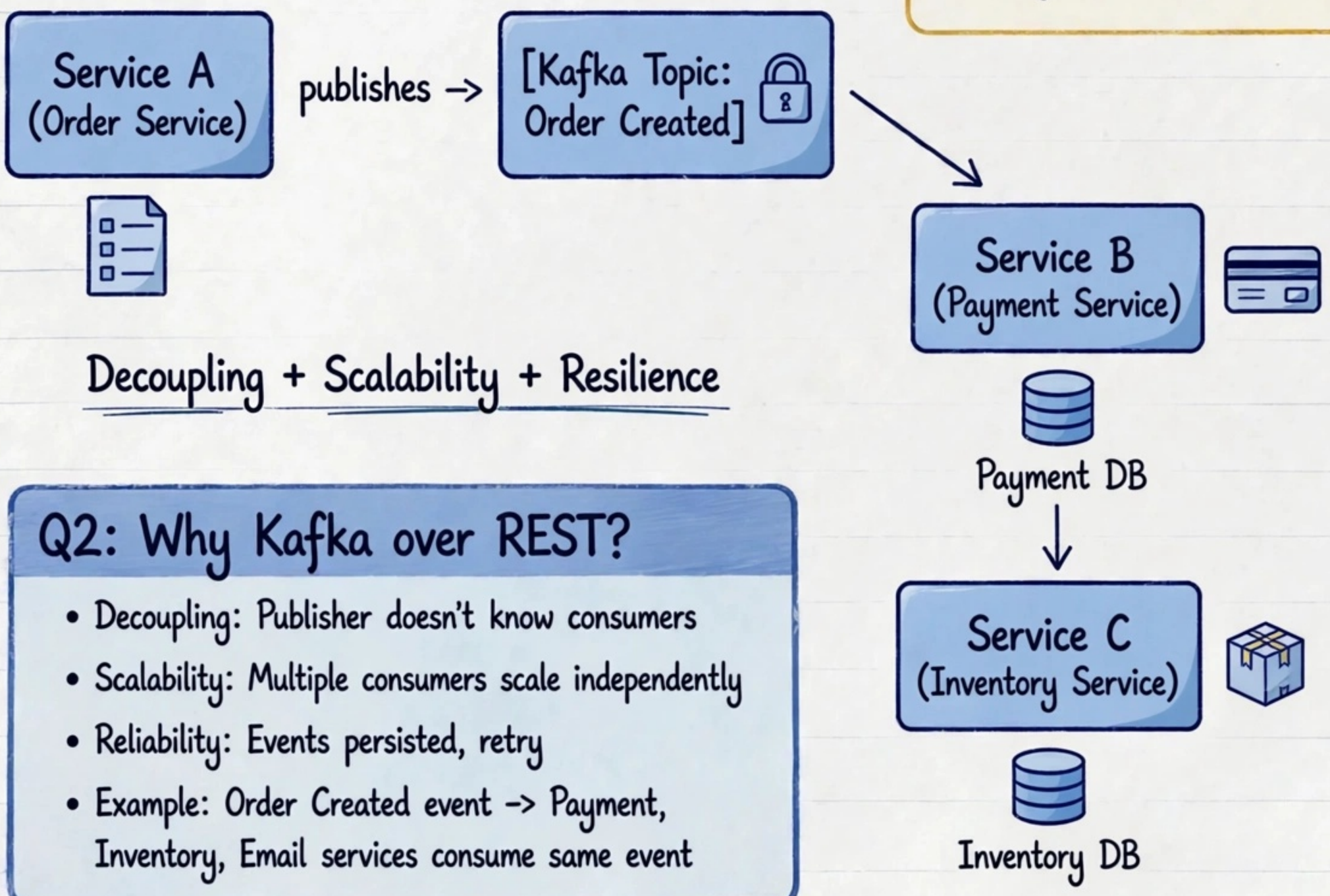
Kafka / RabbitMQ Q&A - Real Production Scenarios

Q1: What is Event-Driven Architecture?

Ans: Async communication pattern where services communicate via events. Service A publishes event to Kafka Topic / RabbitMQ Exchange, Service B, C consume independently. No direct REST call.

Key points you'll learn:

1. Event vs Message
2. Kafka Topic, Partition, Consumer Group
3. At-least-once vs Exactly-once
4. Retry + DLQ pattern



Decoupling + Scalability + Resilience

Q2: Why Kafka over REST?

- Decoupling: Publisher doesn't know consumers
- Scalability: Multiple consumers scale independently
- Reliability: Events persisted, retry
- Example: Order Created event -> Payment, Inventory, Email services consume same event

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

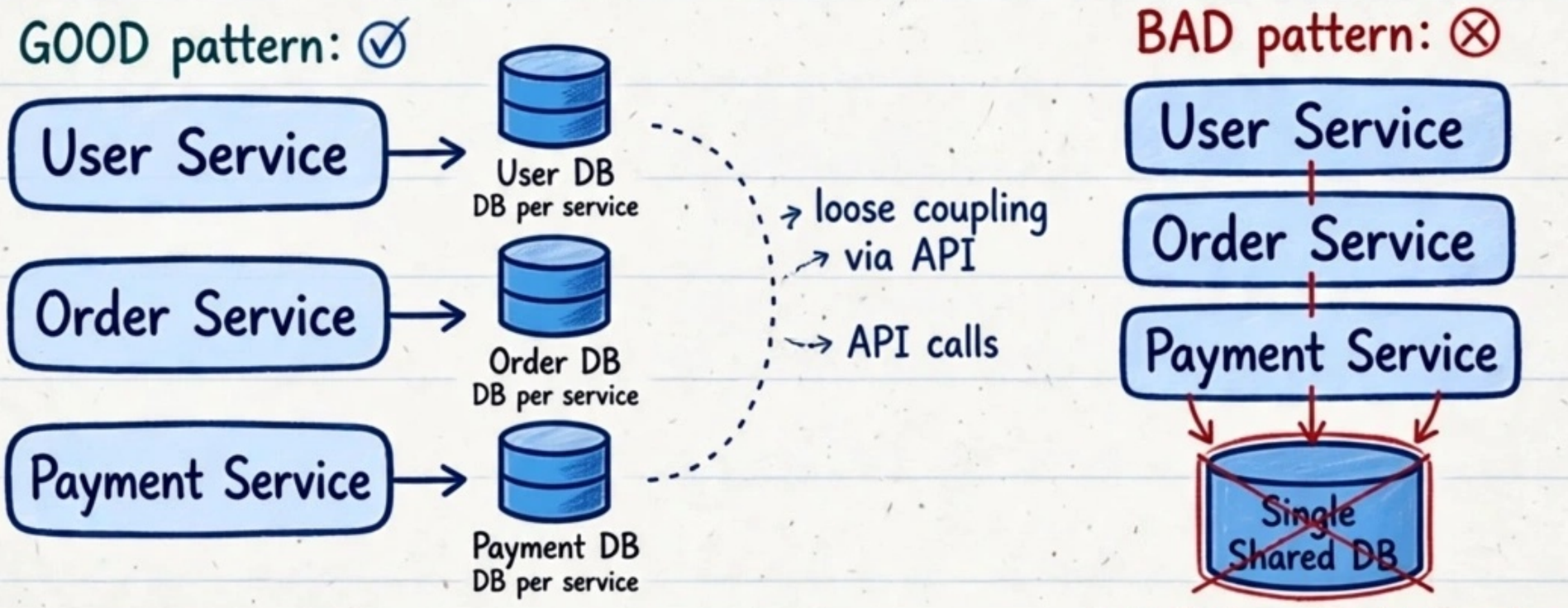
DATABASE PER SERVICE PATTERN Interview Q&A

(Database per Service vs Shared DB) Real Production Q&A

Q1: Should each microservice have its own database?

A: Pattern: Database per Service - each service owns its data, no direct access to other service's DB. Polyglot persistence possible.
Vs Shared DB is anti-pattern - causes tight coupling, scaling issues, single point of failure.
Use SAGA, API Composition, CQRS for cross-service data needs.

Database per Service Visual:



Communication: No direct DB access • Via API • SAGA for transactions

- DB per Service: isolation + autonomy + independent scaling + team ownership •
- Shared DB = anti-pattern: coupling, risky deployments, schema conflicts •
- Cross-service queries: API Composition, CQRS, SAGA handles transactions •

Why DB per Service?:

- Loose coupling, no shared schema
- Each service scales independently
- Polyglot: SQL for User, NoSQL for Product etc
- Failure isolation

How to Handle Cross-Service Data?:

1. API Composition - aggregate via API calls
2. SAGA Pattern for distributed tx
3. CQRS - separate read model
4. Event-Driven replication via Kafka
5. Materialized View Pattern

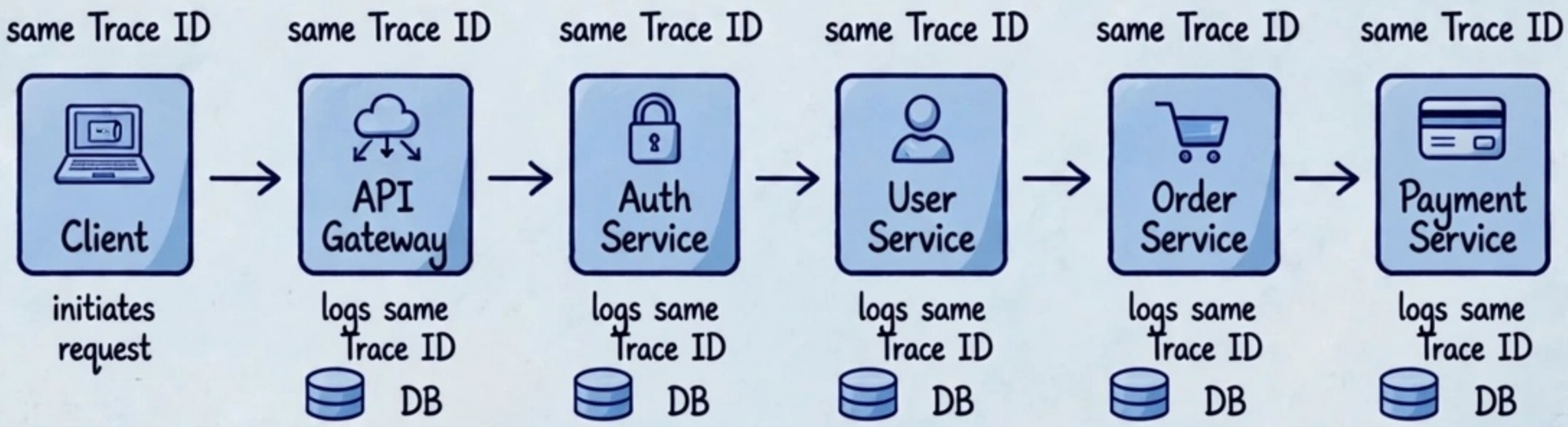
Comment "MICRO PDF"
I'll send you the full Q&A PDF link!

DISTRIBUTED TRACING + LOGGING Q&A

How to debug a request that touches 10 services?

Trace ID: abc-xyz-123 propagated via HTTP Header (X-Trace-ID)

Request Flow with Trace ID Propagation Visual:



Logs → ELK / Loki
Centralized Logs: all logs shipped here

Traces → Jaeger / Zipkin
Visual timeline + spans per service

Metrics
→ Prometheus + Grafana Dashboards & alerts

- What is Trace ID? Unique ID attached to request at API Gateway, passes to all downstream services
- Why needed? Without it logs are scattered across 10 services, impossible to correlat Span (Auth span, User span..
- Centralized Logging: All services log with Trace ID → shipped to ELK (Elasticsearch Logstash Kibana) / Loki
- Distributed Tracing Tools: Jaeger / Zipkin visualizes full request timeline + latency per service
- Observability trio: Logs (ELK) + Traces (Jaeger) + Metrics (Prometheus + Grafana) = full observability

Why Learn This?:

- Most asked in system design + backend interviews
- Debug production issues 10x faster
- Essential for microservices observability
- Every senior interview asks ELK, Jaeger, Trace ID

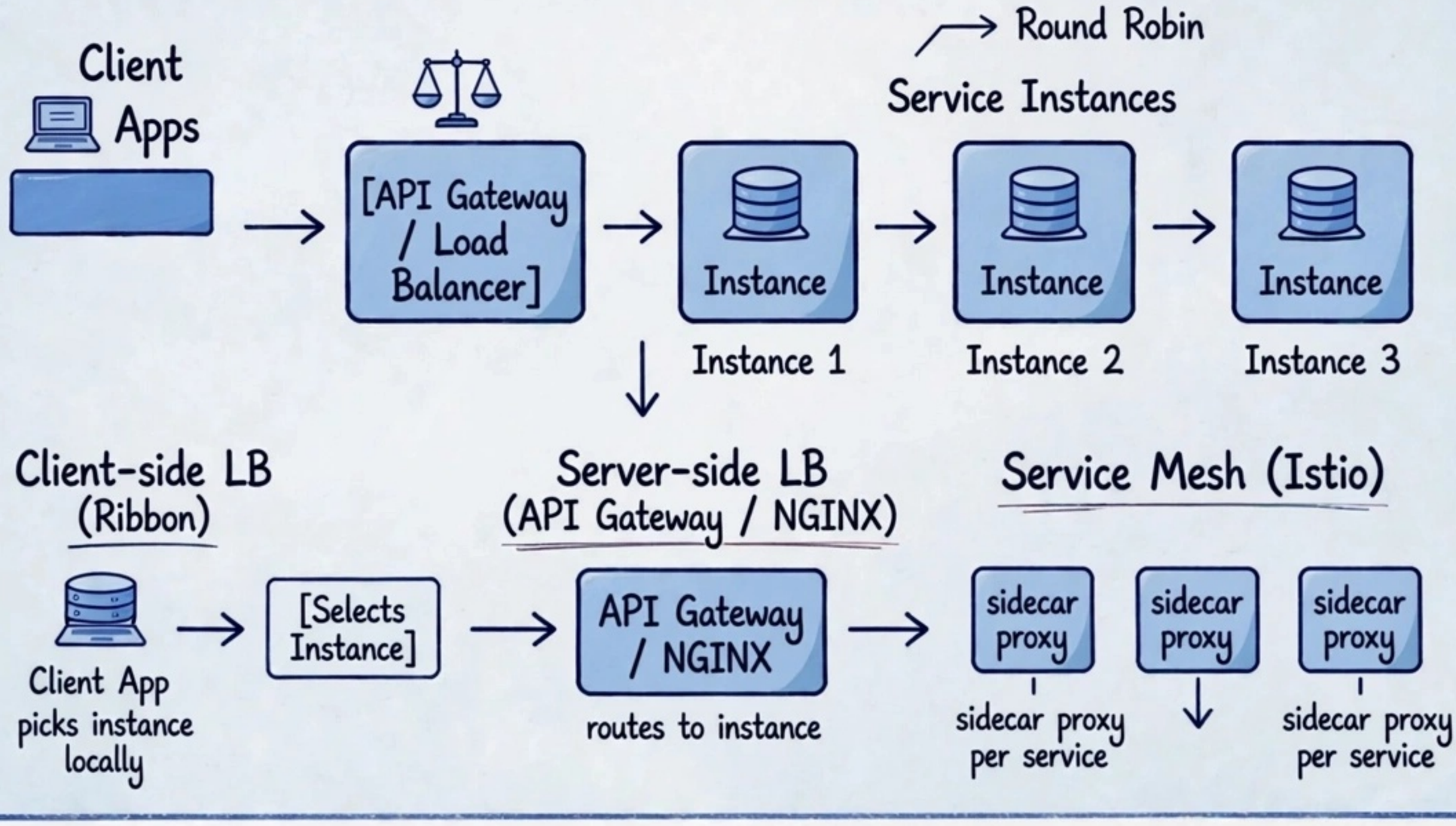
In production: API Gateway generates Trace ID → propagates → Central ELK search by Trace ID → Jaeger timeline

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

LOAD BALANCING IN MICROSERVICES INTERVIEW Q&A visual

Client Side vs Server Side vs Service Mesh – Production Ready Patterns

Load Balancing Types & Flow:



K8s Service = Load Balancer

Algorithms: Round Robin • Least Connections • IP Hash • Weighted

Why Need LB?:

- Distribute traffic across 3 replicas
- No single point of failure + scale
- Health checks + auto failover
- K8s Service does kube-proxy LB

Interview Q&A:

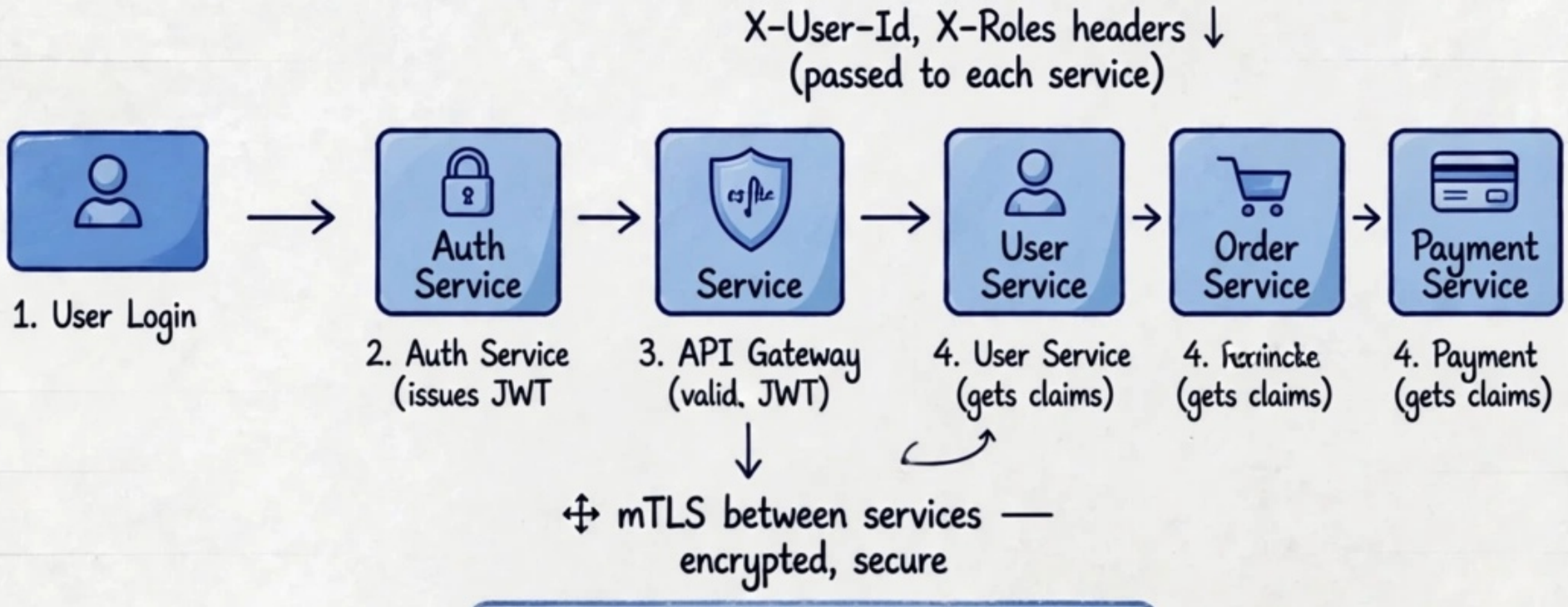
1. What is LB in microservices? Distribute requests across replicas
2. Client-side vs Server-side? Client picks (Ribbon) vs Gateway picks (NGINX)
3. What is Service Mesh LB? Istio sidecar proxy does LB + mTLS
4. K8s Service LB Type? ClusterIP / NodePort / LoadBalancer

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

Q11: HOW TO SECURE MICROSERVICES?
JWT + Oauth2 in Microservices Q&A (Backend Series)

API Gateway + JWT Validation + Passing User Context Downstream

Security Flow – JWT Oauth2 Visual:



JWT Token – Claims:

- user_id
- roles
- exp

- User login: Auth Service authenticates + issues JWT (access + refresh token)
- Oauth2: Authorization Server → JWT with scopes / roles
- API Gateway validates JWT signature, expiry, scopes – rejects 401 if invalid
- Gateway extracts claims + passes user context downstream via headers (X-User-Id)

- Services are stateless: no need to call auth DB, just trust gateway + verify claims
- mTLS between services: service identity, encrypt inter-service traffic
- Bonus: short-lived JWT (15 min) + refresh token rotation + token revocation list

X-User-Id, X-Roles headers ↓
(passed to each service)

📖 Why This Matters for Interview?:

- Top asked in backend interviews 85%
- Never validate JWT in each service – do at Gateway
- JWT = stateless auth, Oauth2 = authorization framework
- Gateway pattern = single auth entry point

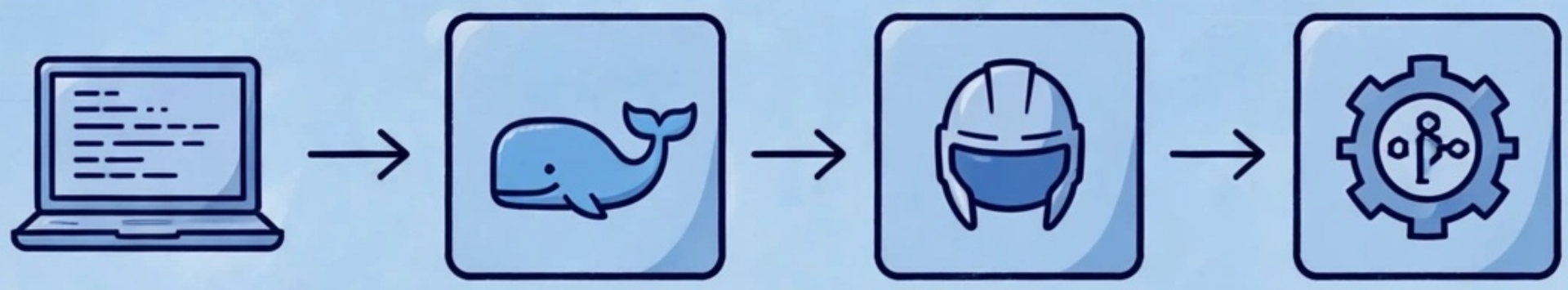
- Communication: HTTPS + JWT + mTLS (service-to-service)

💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

DEPLOYMENT - DOCKER, K8S & HELM Q&A

How to Deploy Microservices in Production? (Backend Interview - Real Scenarios)

Deployment Flow Diagram:



• Pods • Services • Deployments • Ingress

Why Docker + K8s for Microservices?:

- Each microservice = independent Docker container
- Docker ensures consistency dev to prod
- Kubernetes orchestrates scaling, self-healing, service discovery
- Helm = package manager for K8s manifests

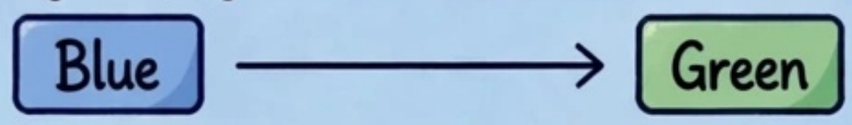
Helm Chart Structure:

- Chart.yaml
- values.yaml
- templates/deployment.yaml
- templates/service.yaml
- templates/ingress.yaml

Deployment Strategies Interview Q&A:

Q: How deploy safely? →

• Canary Deployment (5% = 50% = 100% traffic) Recreate



5% canary → 50% → 100%

💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

SCALING & AUTOSCALING MICROSERVICES vs SERVERLESS

(Backend Series) Autoscaling Strategies + FaaS Comparison Q&A

Scaling in Microservices:

- HPA Horizontal Pod Autoscaler: scales pods based on CPU >70%, memory, custom metrics, queue length (Kafka lag / QPS)
- VPA Vertical Pod Autoscaler: adjusts CPU/memory request
- Cluster Autoscaler: adds/removes nodes when pods pending

pod → pod → pod → pod → pod

HPA → 3 pods → 6 pods

When to Scale What?

- Traffic spike? HPA
- Node full? Cluster Autoscaler
- Queue backlog? KEDA + HPA on lag
- Memory leak? VPA or fix!

Node 1 + Node 3

Cluster Autoscaler adds/removes Node 3

- KEDA: event-driven autoscaler for queue length

Microservices vs Serverless (FaaS) Comparison:

Feature	Microservices (K8s)	Serverless (Lambda/Cloud Function)
Infra:	You manage K8s	No servers, managed by cloud
Scaling:	HPA/VPA manual config	Auto scale instant to zero
Billing:	Pay per VM/pod always on	Pay per invocation + duration (pay per use)
Cold Start:	No cold start	Yes cold start latency 100ms–2s, fix provisioned concurrency
State:	Stateful + Stateless	Stateless only
Timeout:	Long running	Max 15 min (Lambda)
Best for:	Complex long-running business logic	Event-driven, short, spiky workloads

Pros & Cons Serverless

Pros: zero ops, auto scale, cost for spiky ✅

Cons: cold start, vendor lock-in, observability hard, timeout ⚠️

When use What?

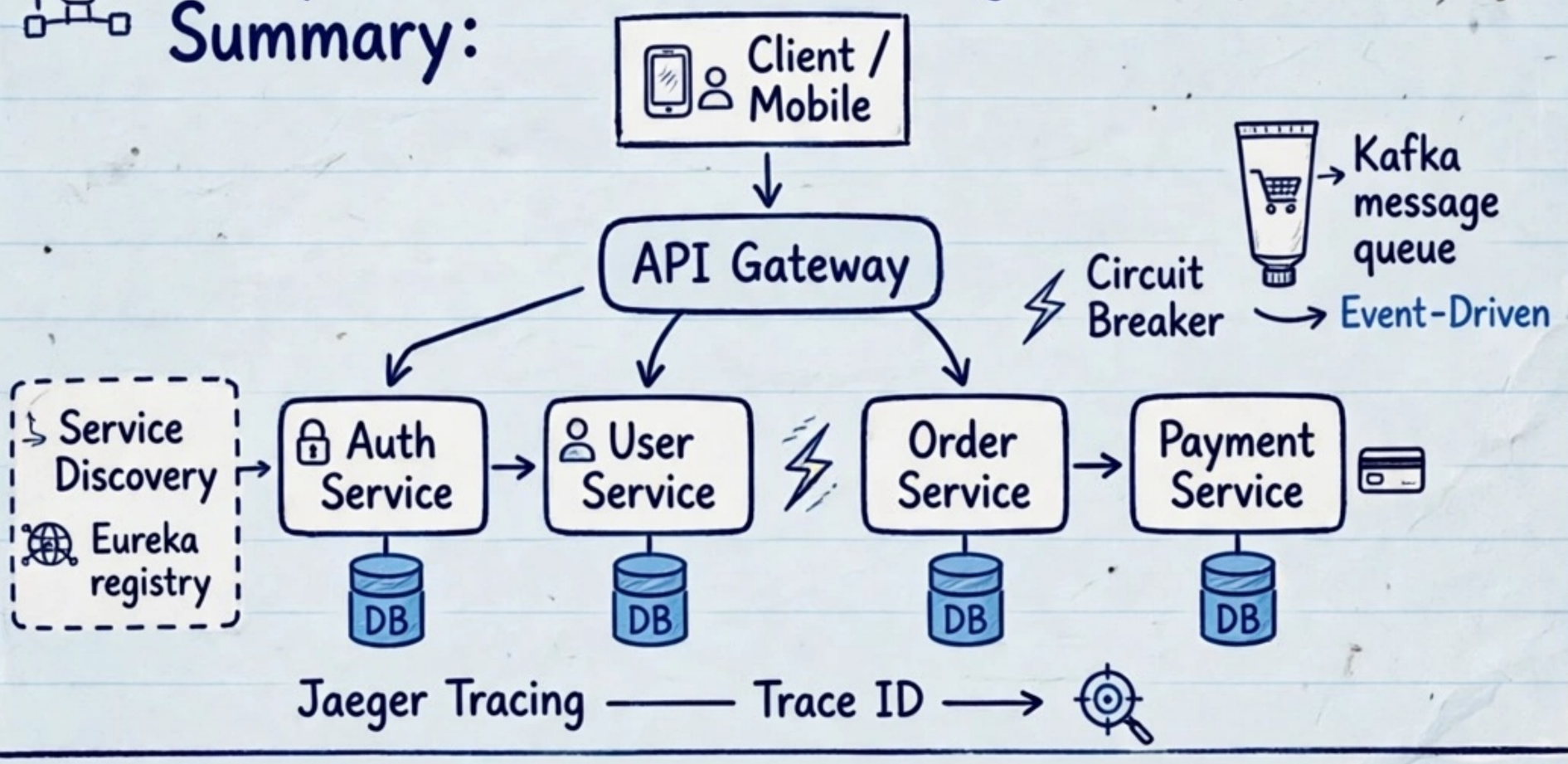
Use Microservices when:
complex domain, long running, need control.
Use Serverless when:
event processing, cron, image thumbnails, webhooks, MVPs.

💬 Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

FINAL CHEATSHEET - TOP 10 INTERVIEW Q&A

Microservices Architecture - All Patterns Revision

Complete Architecture Diagram Summary:



Top 10 Q&A - Must Know:

1. Saga Pattern - How to handle distributed transactions? → Compensating transactions
2. Circuit Breaker - Prevent cascade failures → fallback + Resilience4j /
3. Service Discovery - How services find each other? → Eureka / Consul registry
4. CQRS - Separate Read & Write models
5. API Gateway - Single entry, routing, auth, rate limiting
6. Event Driven - Kafka async communication + eventual consistency
7. DB per Service - Each service owns its database
8. Distributed Tracing - Trace ID, Jaeger / Zipkin
9. Deployment - Docker, K8s, Helm, CI/CD
10. Scaling - HPA autoscaling, independent scaling

Key Takeaways:

- Saga = compensating transactions for atomicity
- Circuit Breaker = fail fast + fallback
- Eureka = heartbeat + registry
- Kafka = async + decoupling
- K8s + Docker = deployment standard

☑ In this page you revised:

- ☑ API Gateway
- ☑ Eureka
- ☑ Circuit Breaker
- ☑ Saga
- ☑ CQRS
- ☑ Kafka
- ☑ DB per service
- ☑ Tracing
- ☑ K8s

Comment "MICRO PDF" and I'll send you the full Q&A PDF link!

@codeera.tech → Follow for more system design tips!